

Tập 5: Cuộc chuyển giao Ingress & Gateway API

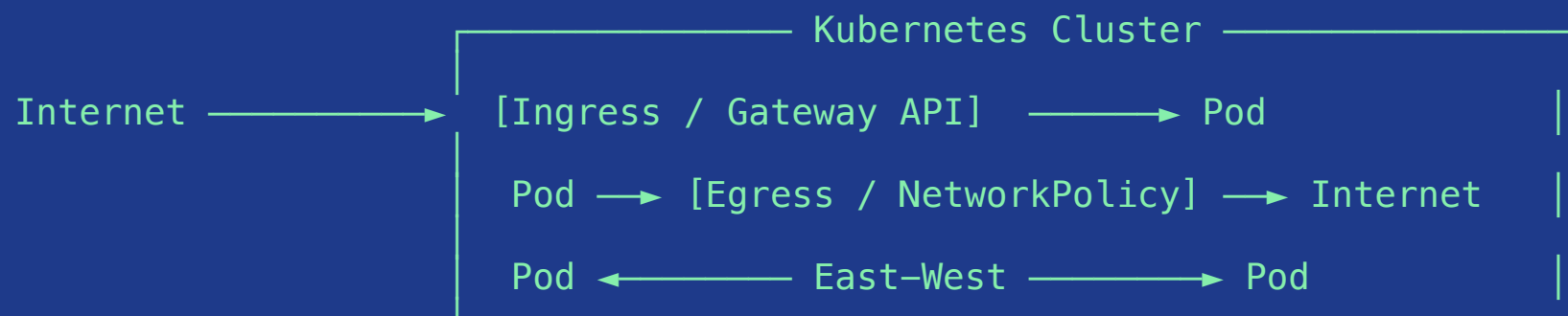
Lý thuyết: ingress-nginx nghỉ hưu & kiến trúc Role-oriented của Gateway API v1.4

Network Thực Chiến · Series: Kubernetes Networking · Tập 05



Bức tranh toàn cảnh: Traffic trong Kubernetes

Có **3 hướng traffic** cần phân biệt trước khi học Ingress:



Hướng	Cơ chế kiểm soát	Tập học
Vào cluster (Ingress)	Ingress resource, Gateway API	Tập 5 — bài này
Ra ngoài (Egress)	NetworkPolicy egress, Egress Gateway	Tập 6
Pod ↔ Pod (East-West)	Service + NetworkPolicy	Tập 3-4

Tập 5 tập trung North-South VÀO — traffic từ Internet đến đúng Pod.

Vấn đề: Làm sao traffic từ ngoài vào Pod?

Services kiểu `LoadBalancer` và `NodePort` hoạt động ở **Layer 4** (TCP/UDP):

```
Client → NodePort :30080 → Service → Pod  
# Chỉ biết IP:Port, không biết gì về HTTP Host, Path, Header
```

Với ứng dụng Web hiện đại, bạn cần **Layer 7 routing**:

- `app.example.com/api` → Service A
- `app.example.com/admin` → Service B
- `beta.example.com` → Service C (Canary 10% traffic)

Giải pháp: Ingress (cũ) và Gateway API (mới).

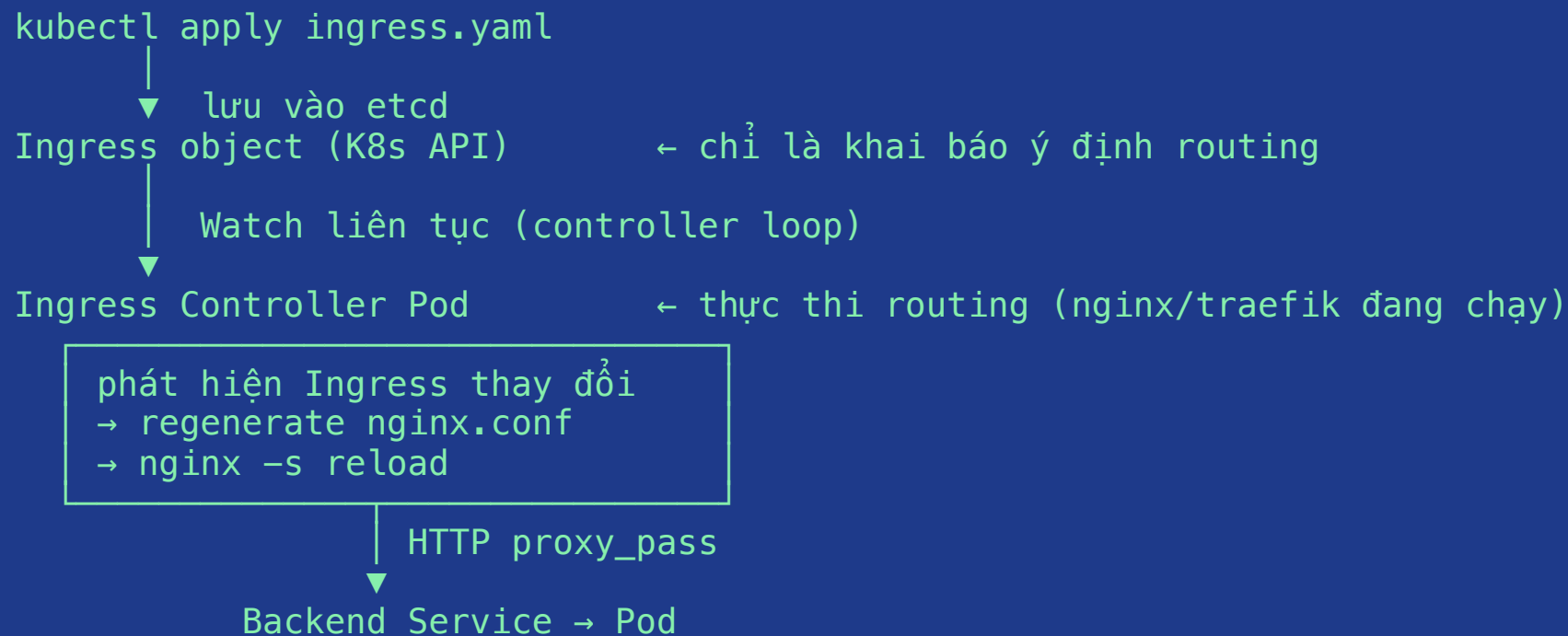
Ingress: Giải pháp cũ (Legacy)

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: / # ← Config qua annotation!
spec:
  rules:
    - host: app.example.com
      http:
        paths:
          - path: /api
            backend:
              service:
                name: api-service
                port:
                  number: 8080
```

Vấn đề cốt lõi: Tính năng nâng cao phải nhét vào **annotation** → không chuẩn hóa, mỗi controller (nginx, traefik, haproxy) dùng annotation khác nhau.

Ingress hoạt động như thế nào?

Ingress resource $\neq$ **Ingress Controller** — đây là 2 thứ hoàn toàn khác nhau:



⚠️ Không cài Ingress Controller → `kubectl apply ingress.yaml` thành công, nhưng **traffic không đi đâu cả**.

ingress-nginx đã chính thức nghỉ hưu

Project `kubernetes/ingress-nginx` đã **archived** ngày **24/3/2026** — repo read-only, không còn nhận PR hay release mới. Lý do:

1. **Mô hình phân quyền thiếu rõ ràng:** Dev và Ops đều phải chỉnh sửa cùng 1 Ingress object.
2. **Annotation hell:** Hàng trăm annotation không chuẩn hóa, vendor-lock-in.
3. **Không hỗ trợ traffic splitting chuẩn:** A/B testing, Canary phải dùng annotation hack.
4. **Gateway API đã GA** và giải quyết được tất cả vấn đề trên.

| ingress-nginx đã dead. Migration sang **Gateway API** không còn là "khuyến nghị" — đây là bắt buộc.

Gateway API: Kiến trúc Role-oriented

Gateway API phân chia trách nhiệm thành 3 tầng rõ ràng:

```
Infrastructure Provider (Cluster Admin)
└─ GatewayClass ← "Nhà sản xuất bộ cân bằng tải"
    (Cilium GatewayClass, NGINX GatewayClass...)

Platform Team (Ops/Infra)
└─ Gateway ← "Cổng vào cluster", cấu hình TLS, port
    (Lắng nghe port 443, gắn cert TLS)

Application Team (Dev)
└─ HTTPRoute ← "Luật điều hướng traffic"
    (Path /api → Service A, Host beta → Service B)
```

GatewayClass: Định nghĩa "Loại" Load Balancer

```
apiVersion: gateway.networking.k8s.io/v1
kind: GatewayClass
metadata:
  name: cilium
spec:
  controllerName: io.cilium/gateway-controller
```

Tương tự như `StorageClass` cho PVC, `GatewayClass` định nghĩa **implementation** nào sẽ xử lý Gateway.

Ai implement Gateway API?

Gateway API Spec (K8s SIG Network – chuẩn chung)	
— Envoy Gateway	← CNCF project, self-contained, Lab 1.5 này dùng
— Cilium	← khi dùng Cilium CNI (Module 2 sẽ học)
— NGINX GF	← từ F5/NGINX Inc.
— Traefik	← phổ biến trong homelab/community
— Istio	← service mesh, L7 đầy đủ nhất

Cùng 1 `HTTPRoute` YAML — chạy được trên **bất kỳ implementation nào** ở trên.

Gateway: Cổng vào cluster (Ops quản lý)

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: prod-gateway
  namespace: infra      # ← Ops team quản lý namespace này
spec:
  gatewayClassName: cilium
  listeners:
    - name: https
      port: 443
      protocol: HTTPS
      tls:
        certificateRefs:
          - name: prod-tls-cert    # ← TLS cert do Ops quản lý
```

HTTPRoute: Luật routing (Dev tự quản lý)

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: api-route
  namespace: my-app      # ← Dev team quản lý namespace này
spec:
  parentRefs:
    - name: prod-gateway
      namespace: infra
  hostnames: ["app.example.com"]
  rules:
    - matches:
        - path:
            type: PathPrefix
            value: /api
      backendRefs:
        - name: api-service
          port: 8080
          weight: 90      # ← Traffic splitting chuẩn!
        - name: api-canary
          port: 8080
          weight: 10
```

ReferenceGrant: Cross-namespace an toàn

HTTPRoute ở `my-app` muốn dùng Gateway ở `infra` → **bị chặn mặc định**:

```
# ❌ Chỉ có HTTPRoute này → status: Accepted=False
spec:
  parentRefs:
    - name: prod-gateway
      namespace: infra  # ← cross-namespace reference
```

Ops team phải tạo `ReferenceGrant` trong namespace `infra`:

```
apiVersion: gateway.networking.k8s.io/v1beta1
kind: ReferenceGrant
metadata:
  name: allow-my-app
  namespace: infra  # ← phải nằm trong namespace CỦA Gateway
spec:
  from:
    - group: gateway.networking.k8s.io
      kind: HTTPRoute
      namespace: my-app  # ← cho phép từ namespace này
  to:
    - group: gateway.networking.k8s.io
      kind: Gateway
```

Debug Gateway API: Status Conditions

Ingress lỗi → `kubectl describe ingress` → xem **Events**

Gateway API lỗi → xem `status.conditions` trên từng object:

```
# Kiểm tra Gateway
kubectl get gateway prod-gw -o yaml | grep -A 8 "conditions:"
# Accepted: True      - controller đã nhận Gateway
# Programmed: False   - bare metal không có External IP (bình thường!)

# Kiểm tra HTTPRoute
kubectl get httproute api-route -o yaml | grep -A 8 "conditions:"
# Accepted: True      - gateway đã chấp nhận route này
# ResolvedRefs: True  - backend services tìm thấy
```

Condition	False → Nguyên nhân thường gặp
<code>Accepted=False</code>	Sai <code>parentRef</code> hoặc thiếu ReferenceGrant
<code>ResolvedRefs=False</code>	Sai tên service, sai port, sai namespace
<code>Programmed=False</code>	Data plane chưa sync hoặc không có External IP

So sánh: Ingress vs Gateway API

Tiêu chí	Ingress	Gateway API
Phân quyền	Một object, ai cũng chỉnh	Tách rõ: Ops (Gateway) / Dev (HTTPRoute)
Traffic splitting	Annotation hack	Native <code>weight</code> field
gRPC, TCPRoute	Không hỗ trợ chuẩn	Hỗ trợ chuẩn
Extensibility	Annotation	Policy Attachment (chuẩn hóa)
Trạng thái	Legacy, EOL	GA từ K8s v1.28, v1.4 đang phát triển

Tổng kết Tập 5

Khái niệm	Tóm tắt
Ingress	L7 routing cũ, annotation-based, EOL tháng 3/2026
GatewayClass	Định nghĩa implementation (Cilium, NGINX...)
Gateway	Cổng vào cluster, do Ops quản lý, gắn TLS
HTTPRoute	Luật routing, do Dev quản lý, hỗ trợ traffic splitting

👉 Chuyển sang Lab 1.5

Mở file `lab-guide.md` trong thư mục `1.5/` để thực hành:

- Cài ingress-nginx và cấu hình routing bằng Ingress cổ điển
- Xem file `nginx.conf` được tự động sinh ra
- Migrate sang Gateway API với Cilium làm controller